

基于沙盒环境的开源深度研究 智能体评测基准

*Deep-Research Arena: A Sealed-Sandbox, Truthfulness-Gated
Benchmark for Open-Source Deep-Research Agents*

作者: LIU Yibo
学号: _____
学院: _____
专业: _____
指导教师: _____
答辩日期: 2026 年 5 月

学位论文答辩稿

摘要

本文构建并验证了一套面向开源深度研究 (Deep Research, DR) 智能体的可复现评测基准 **Deep-Research Arena**。该基准用三个本地化的密封服务替代开放互联网作为信息来源: 运行于 `localhost:7770` 的 Magento 电商门店、`localhost:9999` 的 Postmill 论坛、`localhost:8090` 的 Kiwix 离线维基百科镜像。两个粘合层服务, 即 `localhost:8081` 上 Tavily 兼容的搜索 shim 与 `localhost:8088` 上 OpenAI 兼容的大模型代理 `ds_proxy`, 是智能体进程访问搜索与模型流量的唯一通道。

本文将 13 个主流开源 DR 框架接入该沙盒, 其中 9 个框架在 56 个跨站点深度任务上产生足够多的有效运行, 进入头部排名表。每份智能体报告在 7 个验证维度上打分: 其中 URL 可达性、引文文本匹配、claim NLI 蕴含三项为确定性的真实性探针, URL 覆盖、checklist、引文对齐、表达与分析深度等四项则将确定性的结构检查与 DeepSeek V4-flash 在零温度下的 LLM 判官相结合。三个复合分数共存: `composite_v1` 为加性形式, 作为基线; `composite_v2_truthful` 用可达性作为乘性真实性门, 是正式排名指标; `composite_v3` 进一步引入对齐、深度、表达三个维度, 并将硬门松弛为 $\max(0.1, \text{reach})$ 的下界。

本文最重要的发现是: 真实性门会反转朴素的排名。在 `composite_v1` 下, GitHub 社区热度最高的 `gpt-researcher` 框架以 1269 Elo 居首; 在 `composite_v2` 下, 同一框架掉至第 5 名 (Elo 866), 而具有显式 URL 去重与引文校验流程的 `camel-ai` 升至第 1 名 (Elo 1481)。流畅的文本表达并不等同于有据可查的证据, 而把两者放在同一指标上打分会奖励错误的行为。本文的贡献是把两者分离的设计选择、能够在普通硬件上复现的实现、以及来自 283 份打分文件、9 个框架、56 个任务的实证证据, 证明这一分离在实践中是显著的。

关键词: 深度研究智能体; 评测基准; 沙盒环境; 真实性门; Bradley-Terry 排名; Elo; LLM 判官; ALCE; 复现性

Abstract

This thesis builds and validates a reproducible benchmark, **Deep-Research Arena**, for open-source deep-research (DR) agents. The benchmark substitutes the live internet with three sealed services: a Magento storefront on `localhost:7770`, a Postmill forum on `localhost:9999`, and a Kiwix offline-Wikipedia mirror on `localhost:8090`. Two glue services, a Tavily-compatible search shim on `localhost:8081` and an OpenAI-compatible LLM proxy `ds_proxy` on `localhost:8088`, are the only routes by which an agent’s process can reach search or model traffic.

Thirteen open-source DR frameworks are instrumented to route both kinds of traffic through these endpoints; nine produce enough valid runs across 56 cross-site deep tasks to enter the headline Bradley-Terry ranking. Each agent report is scored on seven verification pillars, three of which (URL reachability, quote match, claim NLI) are deterministic truthfulness probes and four of which (URL coverage, checklist, citation alignment, presentation and analysis depth) blend deterministic structure checks with DeepSeek V4-flash judges at temperature zero. Three composite scores coexist: `composite_v1` is additive, `composite_v2_truthful` multiplies a reachability gate over the quality term and is the headline metric, and `composite_v3` extends the multiplicative form with citation alignment, depth and presentation while replacing the hard gate with a $\max(0.1, \text{reach})$ floor.

The single most consequential finding is that the truthfulness gate inverts naive ranking. Under `composite_v1` the GitHub-popular `gpt-researcher` framework tops the leaderboard at Elo 1269. Under `composite_v2` the same agent drops to fifth place at Elo 866, and `camel-ai`, which has explicit URL-deduplication and citation-validation passes, rises to first at Elo 1481. Fluent prose is not the same thing as cited evidence, and a benchmark that scores them together rewards the wrong behaviour. The contribution of this thesis is the design choice that separates them, the implementation that runs reproducibly on commodity hardware, and the evidence from 283 scored runs that the separation matters.

Keywords: deep research agents; evaluation benchmark; sealed sandbox; truthfulness gate; Bradley-Terry ranking; Elo; LLM-as-judge; ALCE; reproducibility

目录

1	绪论	1
1.1	研究背景与动机	1
1.2	现有工作与不足	2
1.3	本文贡献	2
1.4	论文组织结构	3
2	系统总体设计	3
2.1	系统架构总览	3
2.2	沙盒服务设计	5
2.2.1	Magento (shopping, localhost:7770)	5
2.2.2	Postmill (reddit, localhost:9999)	6
2.2.3	Kiwix (Wikipedia, localhost:8090)	6
2.2.4	搜索 shim (localhost:8081)	6
2.2.5	DS proxy (localhost:8088)	7
2.3	任务设计	8
3	13 个开源 DR 框架适配	9
3.1	适配范式	9
3.2	camel-ai (in-process)	9
3.3	smolagents (in-process)	10
3.4	gpt-researcher (subprocess via clean runner)	10
3.5	langchain-open_deep_research (in-process)	10
3.6	ii-researcher (subprocess)	11
3.7	dzhng (subprocess via Node HTTP API)	11
3.8	flowsearcher-ds (in-process, 自研)	11
3.9	deerflow (subprocess via clean runner)	11
3.10	ldr (subprocess via clean runner)	12
3.11	qx-agents (subprocess via clean runner)	12
3.12	tongyi-dr (subprocess via clean runner)	12
3.13	deepagents (subprocess via clean runner)	13
3.14	storm + co-storm (subprocess via clean runner)	13
3.15	框架汇总	13
4	评分体系设计	14
4.1	七维评分概览	14
4.2	url_coverage	15

4.3	url_reachability	15
4.4	quote_match	15
4.5	claim_nli	16
4.6	checklist	16
4.7	citation_alignment	17
4.8	analysis_depth 与 presentation	17
4.9	复合分数 composite_v1 / v2 / v3 的推导	18
5	排名方法学	19
5.1	Bradley-Terry 模型	19
5.2	Bootstrap 置信区间	19
5.3	置换 p 值显著性检验	20
5.4	退化样本过滤规则	20
6	实验结果	20
6.1	实验设置	20
6.2	最终排行榜 (composite_v2_truthful)	21
6.3	相邻名次显著性	21
6.4	关键发现	22
6.4.1	真实性门反转朴素排名 (truth-gate inversion)	22
6.4.2	多智能体协作架构在本沙盒上占优	23
6.4.3	社区热度 vs 实测排名分歧	23
6.4.4	框架级失败模式	24
7	讨论	25
7.1	工作的局限性	25
7.2	工程实现要点	26
7.3	公平性审计	26
8	总结与展望	26
8.1	工作总结	26
8.2	未来研究方向	27
A	复现命令	30
B	术语表	31

1 绪论

1.1 研究背景与动机

深度研究 (Deep Research, DR) 智能体是一类阅读研究问题、上网搜索证据、抓取页面并撰写带引文报告的智能体系统。我们希望评估的能力是多面的: 智能体是否找到了正确的信息源, 是否忠实地引用了这些信息源, 是否做了跨源综合而非仅复述其中一个, 是否承认了自身的局限。直觉上, 衡量这些能力的自然环境就是开放互联网, 目前最被广泛引用的几个 DR 基准, GAIA [1]、BrowseComp [2]、AssistantBench [3]、BrowserGym [4], 也都是在开放互联网上运行的。本论文的核心命题是: 对于以**科学测量**智能体能力 (而非以**产品评测**) 为目标的研究而言, 开放互联网恰恰是错误的环境。

第一个问题是**漂移**。一周前还能支撑某项结论的页面今天可能 404, 昨晚可能被人修改过, 今早可能被搜索引擎重排。两位研究者在相隔一个月时跑「同一个」基准, 得到的数字会不同, 且双方都无法证伪对方。基准本身因此变得不可证伪。GAIA 通过把答案锚定在缓慢变化的事实上部分缓解了一个问题, 但它无法锚定**检索路径**: 2024 年智能体找到了支撑页面, 并不意味着 2026 年同样的查询会返回同样的页面。

第二个问题是**真实性打分在开放网上几乎不可行**。真实性打分至少要回答三个子问题: 智能体给出的 URL 能否解析、页面是否包含被引用的文本、被引用的页面是否真的支撑周围的论断。在开放网上, 这三个探针的每一次, 都是与地理路由、登录墙、限流、CDN 抖动、页面静默改版的赌博。从智能体所在主机返回 200 的 URL, 从打分主机看可能就是 403。ALCE [7] 通过预先固定语料并对语料打分回避了这一问题; 本论文把同样的做法推广到了多语料的交互式沙盒。

第三个问题是开放网基准把**产品评测**与**能力测量**混在了一起。当 gpt-researcher (GitHub 25.7k stars) 在某个开放网基准上拿到高名次时, 有两种读法: 其一是它的智能体架构好, 其二是它的提示词对开放网页面调得很顺、检索流水线针对真实的 Google/Bing 排名做了精细工程化、且框架作者投入了两年时间在「论文到产品」之间的差距上。两者都是合理意义上的进步, 但我们声称要测量的只是其中第一种。一个固定搜索 shim、冻结页面语料的沙盒基准可以主动屏蔽掉第二种读法: 不论排名如何变化, 变化都得归因于**在同一检索基底上的智能体逻辑**。这是 ALCE [7]、FactScore [8]、WebArena [5] 都先冻结基底再打分的原因。

第四个 (较小的) 问题是, 开放网上的评测**昂贵**。每次运行都要拉真实页面、撞真实限流、烧真实 token, 而很多页面下月就不在了。沙盒可以让我们在评测主机上 15 分钟内复跑一个智能体在一个任务上的运行、确定性打分、随时复现。

这场交易的本质是: 用**真实感**换取**可复现性**。沙盒的覆盖面注定不如开放网 (本文三个语料合计大约 2000 个商品页、若干万 Reddit 评论、若干 Wikipedia 文章)。但被牺牲的覆盖面换来了: 每一个真实性探针都可以在 $O(1)$ 时间内对冻结的真相执行, 且 404 与网络抖动之间不再需要主观判断。本论文接受这场交易。

1.2 现有工作与不足

DR 基准。GAIA [1] 是最早被广泛引用的 DR 类基准,把评测建模为「智能体对开放网 + 答案字符串打分」。BrowseComp [2] 沿着同一条轴线扩展为更难的多步任务与更细的评分细则,但基底仍是开放网。AssistantBench [3] 转向长程任务与部分得分,基底仍是开放网。BrowserGym [4] 提供了统一的浏览器环境 API,可以接到开放网或 HAR 回放,在精神上与本工作最相近,但它只规定环境而不规定打分栈。

沙盒化智能体评测。WebArena [5] 是本工作最直接的祖先:一个密封的 Magento + GitLab + Reddit + Map + Wikipedia 装置,在此上测试网页导航智能体并以任务完成度打分。本基准沿用 WebArena 的沙盒化哲学,沿用了其 Magento 与 Postmill 实例,但把「任务完成位」换成了「多维度报告质量分」。AgentBench [6] 在 8 个环境上提供更广义的沙盒智能体评测,但其面向单任务成功,不是面向多源综合。

引文忠实度。ALCE [7] 提出了每引文的精确率/召回率框架:被引用的 URL 是否真的支撑了相邻论断?本论文的 `citation_alignment` 维度直接采用了这一框架。FactScore [8] 把长文本分解为原子事实并对每个事实在知识库上打分;本论文借鉴了其多维度的想法,但保持验证面更紧:三个确定性探针 + 四个 LLM 评分细则,而非每个原子事实一次判断。

RAG 与有据生成。RAGAS [9] 为检索增强生成提出了一组自动指标(忠实度、答案相关性、上下文相关性)。本文的 `claim_nli` 维度即此构造,但执行对象是每任务预先准备的「黄金引文片段」而非运行时检索到的上下文。

LLM 作为判官。零温度 LLM 判官的范式自 MT-Bench [10] 与 Chatbot Arena [11] 以来已成标准。本工作采用了其细则分解变体(将单一 Likert 拆为多个二值),并辅以确定性的「锚定关键词交叉检查」:当相关 URL 不可达时,将 LLM 判官的 PASS 判决降为 FAIL。

基于成对比较的排名。Chatbot Arena [11] 在人工标注的成对胜负上使用 Bradley-Terry [12] / Elo [13] 模型。本论文使用同一模型,但由复合分数合成对战,理由是:在小智能体集合(头部 9 个)与冻结任务集(56 个)之下,成对比较是比原始平均复合分更稳定的聚合方式(详见第五章)。

本基准的独到之处。据笔者所知,以下三点组合在文献中尚未出现:其一,由单一 Tavily 兼容搜索 shim 粘合的多源密封沙盒(Magento + Postmill + Kiwix);其二,乘性的真实性门(`composite_v2`),把 URL 可达性变成了不可妥协的条件,捏造 URL 的智能体无论文笔多好都将拿到 0 分;其三,完整的可复现表面:沙盒 + shim + `ds_proxy` + 黄金池 + 打分代码全部冻结于仓库,任何审稿人拿到快照即可从零重建论文中的每一个 Elo 数字。

1.3 本文贡献

本论文的贡献可归纳为五点:

1. **密封的多源沙盒**。由 Magento (电商)、Postmill (论坛)、Kiwix (百科) 三个真实 Web 应用构成的沙盒, 加上 Tavily 兼容的搜索 shim 与 OpenAI 兼容的 `ds_proxy`, 使得「真实智能体在真实工具栈上」的设置在单机上完整重现, 且智能体进程无任何真实互联网出口。
2. **七维评分体系**。三个确定性真实性探针 (URL 覆盖、URL 可达、引文文本匹配) + 四个 LLM 判官细则 (checklist、引文对齐、表达、分析深度), 全部带有降级保护, 可单独审计也可合成为复合分。
3. **乘性真实性门 `composite_v2`**。用 URL 可达性作乘性门, 把「捏造引文」从「扣分项」升格为「归零项」。这是引出本文头号发现 (真实性门反转排名) 的设计选择。
4. **基于 Bradley-Terry 的 Elo 排名 + Bootstrap 置信区间 + 置换显著性检验**。用 1000 次自助重采样估计 95% CI, 用 1000 次置换检验给出相邻名次的 p 值, 把「智能体 A 强于智能体 B」从经验观察上升为可检验的统计命题。
5. **13 个开源 DR 框架的统一接入层**。三种适配范式 (进程内 monkey-patch、子进程驱动、Node HTTP API), 每个框架都被记录了类型、虚拟环境、入口文件、踩过的坑; 头部 9 个框架累计产生 283 份打分文件 (头部排名所用) 进入 Elo。

1.4 论文组织结构

本论文共八章。第一章为绪论, 陈述研究背景、现有工作与本文贡献。第二章给出系统总体设计, 包括架构总览、沙盒服务的五个组件, 以及任务的设计原则与构造。第三章逐一介绍 13 个开源 DR 框架的适配方式与踩过的坑。第四章详细给出评分体系的设计, 从七个维度的定义、公式与代码位置, 到三个复合分数的数学推导。第五章给出排名方法学, 包括 Bradley-Terry 模型、Bootstrap 置信区间与置换显著性检验, 以及退化样本过滤规则。第六章为实验结果, 展示完整的 13 行排名表 (含 Elo、95% CI、胜负平、对战数)、相邻名次显著性表, 以及四个关键发现: 真实性门反转、多智能体协作的优势、社区热度与实测排名的分歧、框架级失败模式。第七章为讨论, 涵盖局限性、工程实现要点、公平性审计。第八章为总结与展望, 概括工作并指出未来方向。

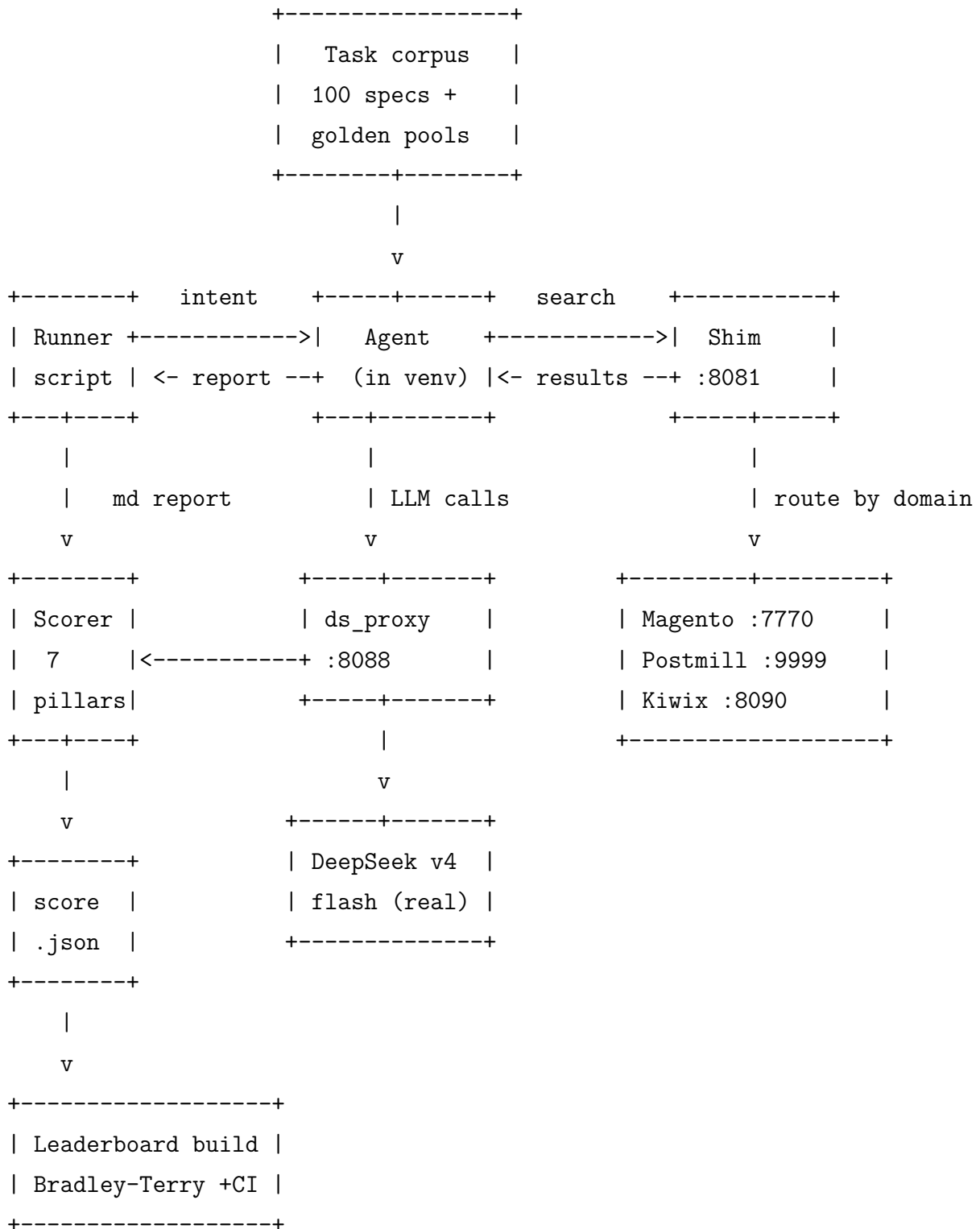
2 系统总体设计

2.1 系统架构总览

整个基准由三个沙盒服务、两个粘合服务、一份任务语料、一套打分栈和每框架一份运行器构成。所有组件运行于一台 Windows + WSL 主机 (`westd`), 所有组件可通过 `localhost` 访问。

智能体进程只能看见 localhost。除搜索 shim 与 LLM 代理外, 网络出口全部被屏蔽。每一次搜索调用都从智能体的虚拟环境出口到 8081 端口; 每一次模型调用都出口到 8088 端口。智能体产出的每一个 URL 都会与三个沙盒 host:port 对作严格比对 (localhost:7770、localhost:9999、localhost:8090); 任何指向其它地址的 URL 都不计入引文覆盖与可达性。这一条规则, 是真实性可被打分的根本原因。

下面的 ASCII 框图给出端到端数据流:



打分器 `scripts/score_deep_answer.py` 读取运行器产出的 Markdown 报告, 计算 7 个维度的分数, 把所有数字写入一份 `<agent>__<task>_matrix.score.json`, 此外不写任何东西。排行榜构造器 `scripts/build_deep_leaderboard.py` 遍历所有 score 文件, 丢弃退化样本, 在 `composite_v2` 上按任务合成成对对战, 拟合 Bradley-Terry 模型并附 Bootstrap CI, 产出 `LEADERBOARD_DEEP.md` 与 `leaderboard_deep.json`。Web 应用 `web/server.py` 实时读盘这两份文件, 无数据库, 使得正在跑的 bulk run 在每次 builder 跑完之后立即可见。

表 1: 系统组件清单

组件	位置	负责
Magento (shopping)	localhost:7770	≈2000 商品页、类目、评论
Postmill (forum)	localhost:9999	论坛帖、评论、子版块层级
Kiwix (Wikipedia)	localhost:8090	离线英文维基 ZIM 镜像
搜索 shim	localhost:8081	Tavily 兼容 API, 索引三个语料
DS proxy	localhost:8088	OpenAI 兼容代理 → DeepSeek V4-flan
运行器分派	run_deep_task.py:1359	将 <code>--agent</code> 映射到 13 个 runner
验证器栈	src/verifiers/	7 个 pillar, 由 <code>score_deep_answer</code> 调用
复合分数	src/scoring/leaderboard_composites.py	v1/v2/v3 的唯一定义来源
排行榜构造	scripts/build_deep_leaderboard.py	读 score JSON, 写 Elo 表

2.2 沙盒服务设计

沙盒围绕三个属性而设计: 智能体可能引用的每一个 URL 都可确定性地解析, 三个语料彼此差异足够大以使跨源任务非平凡, 整套栈在单机上不需要任何实时互联网调用。每个语料都是真实 DR 信息基底「最小可行片段」。

2.2.1 Magento (shopping, localhost:7770)

电商语料是从 WebArena 的 onestopmarket 快照导入的 Magento 2 实例, 大约 2000 个商品页, 横跨三大类目 (Electronics、Home & Kitchen、Beauty & Personal Care) 与数十个叶子类目。每个商品页含 URL slug、价格、星级、评论数、自由文本特征列表、若干用户评论; 头部任务中的「推荐」与「比较」类目就依赖于这些字段。选择 Magento 的理由有三: 它是 WebArena 用的真实电商基底; 商品页 URL 结构稳定地以 `/product/...html` 结尾; 直接复用了 WebArena 的「类目页 → 商品页」浏览模式, 不必重写。两个实践后果:

1. Magento 的 URL 路径大小写敏感 (默认设置)。把路径转小写的框架会自动失去可达性; ldr 的失败模式之一正是这条。

2. 在并行 HEAD 探活下, Magento 偶尔会重置连接。URLReachabilityVerifier._probe (位于 `src/verifiers/url_reachability_verifier.py:55-82`) 因此采用 GET (而非 HEAD), 配合 3 次重试与指数退避。

2.2.2 Postmill (reddit, localhost:9999)

论坛语料同样来自 WebArena 的 Postmill 实例, 包含一批精选「子版」(`/f/technology`、`/f/gadgets`、`/f/AskReddit`、`/f/headphones`、`/f/askscience` 等), 含帖子、评论、点赞数、时间戳。选择 Postmill 的理由是它是 WebArena 自带的开源 Reddit 克隆; URL 结构 (`/f/<forum>/comments/<post_id>/<slug>`) 稳定; 评论层级让智能体能从静态商品页拿不到的「社区情绪」中获取信号。任务中要求智能体将帖子分类为 `praise`、`complaint`、`technical_question`、`comparison`、`purchase_advice` 等类别, 这一切都依赖评论文本的完整。

2.2.3 Kiwix (Wikipedia, localhost:8090)

百科语料是 Kiwix 服务器加载 `wikipedia_en_all_nopic` ZIM 文件构成的, 文章 URL 形如 `localhost:8090/content/wikipedia_en_all_nopic/A/<Article_Title>`。选择 Kiwix 的理由有三: ZIM 文件是英文维基的冻结签名快照; Kiwix 服务器轻量且确定; 维基镜像是用以核验技术性论断 (编解码器、电池、RFC) 的天然「百科基底」。两个实践后果:

1. 文章路径大小写敏感 (`/A/Bluetooth` 可达, `/A/bluetooth` 返回 404)。若干框架默认把路径归一化为小写, 会在维基语料上单独拿到 `reachability=0;ldr` 排名靠后部分原因即此 (其 30 份打分的 `composite_v2` 均值 = 0.029)。
2. 部分框架会输出 `en.wikipedia.org/wiki/<Title>` 形态的 URL, 因为它们的搜索栈拿到的就是开放维基 URL。HTTP 层拦截 (`scripts/run_deep_task.py:107-115`) 会在传输层把这些重写为 Kiwix 前缀; 若无此重写, `langchain-odr` 与 `ii-researcher` 都会输出充满外部 URL 的报告, 可达性归零。

2.2.4 搜索 shim (localhost:8081)

搜索 shim 是 Tavily 兼容的 HTTP 服务, 实现于 `integrations/search_shim/app.py`。对外暴露 `POST /search` 与 `POST /extract` 两个端点, 使用 Tavily 标准的请求/响应 schema。内部将每个查询路由到三个索引之一或多个 (商品标题索引、论坛主题 + 正文索引、维基标题 + 导言段落索引), 合并 top-N, 以 Tavily JSON 形态返回。三条重要性质:

1. **Tavily 兼容即集成契约**。本基准接入的每一个框架都已经支持 Tavily。把每个框架的 Tavily 客户端指向 shim 是一行 `base_url` 配置即可完成的事, 无需写自定义

工具或调提示词。

2. **无任何真实互联网调用。** shim 从不发出对外网的网络调用。它只服务三个索引中不存在的内容。智能体若搜索沙盒不存在的品牌, 会得到空列表, 与 Tavily 对零命中查询的行为一致。
3. **排序稳定。** 搜索结果按固定的 BM25 排序, 而非按时间或会话。第 0 分钟与第 1000 分钟的同一查询返回完全一样的结果。这一性质是每任务黄金池得以作为冻结真相的基础。

shim 是让整个架构生效的关键: 第三章中每一个集成, 本质上都是把某框架的 Tavily 客户端指向 `http://localhost:8081` 的若干行胶水代码。

2.2.5 DS proxy (localhost:8088)

DS proxy 是 OpenAI 兼容的大模型端点, 实现于 `integrations/ds_proxy/app.py`。智能体虚拟环境把它当作 `https://api.openai.com/v1` 来用, 使用相同的 `Authorization: Bearer <key>` 头与相同的 `/chat/completions` JSON 形态。代理把每个请求翻译为 DeepSeek API 调用, 在请求体中注入 `thinking: disabled` (使骨干模型从不在「推理」上花 token), 然后流式回传响应。集中化 LLM 的三条理由:

1. **全基准一个骨干。** 头部排名中每个智能体都跑在同一份 `deepseek-v4-flash` 非推理 checkpoint 上。跨智能体对比因此测量的是智能体架构, 而非骨干强度。未来「跨 LLM 比较」页面 (Web 应用中 `/compare` 已布好路由) 将只换代理的后端, 而不换智能体。
2. **服务端持有密钥。** 智能体只看见 `OPENAI_API_KEY=anything`。真实 DeepSeek 密钥存于代理的环境变量中, 永不进入任何智能体虚拟环境或驱动脚本。这也是新接入的运行器可零配置启动的原因: 它继承自 `scripts/run_deep_task.py:50-53` 的 `OPENAI_BASE_URL` 与字符串 `anything` 作密钥。
3. **单点关闭推理。** 部分框架 (`langchain-odr`、`qx-agents`) 会因为模型名中包含某些子串而静默切换到推理模式。在代理层注入 `thinking: disabled` 拦截每一个请求, 无论上游框架是谁, 保证比较是在同一份非推理 checkpoint 上进行。

代理也充当判官端点。`judge_client.py` 读取 `JUDGE_BASE_URL=http://localhost:8088/v1`, 于是每次 LLM 判官调用 (`checklist`、`citation_alignment`、`presentation`、`analysis_depth`、`claim_nli`) 都打到同一份代理、同一份骨干、`temperature: 0`。LLM 判官的确定性, 依赖于这一温度设置加上 DeepSeek 文档承诺的零温度确定性。

2.3 任务设计

任务语料是 100 个跨站点深度研究任务, 其中 57 个完成了端到端打分 (一个任务 `dr_cross_deep_0044` 在黄金池健全性检查中被剔除)。每个任务是 `data/tasks/deep_research/cross_site_deep/` 下的一份 JSON 文件, 内含自然语言意图、引文政策 (每域最小、总量最小、必须域名列表)、Markdown 规范 (字数/引文数/段落数)、URL 覆盖设置 (黄金池路径、`recall/coverage` 阈值)、URL 可达阈值、黄金三元组文件路径、可选的综合要求, 以及每任务 checklist JSON 的路径。schema 带版本号 (`schema_version: deep-1.0.0`), 便于验证器对新增字段的兼容。

三条任务设计常量:

1. **每任务三个站点。**每个跨站点任务都要求引用全部三个沙盒语料中的页面。`citation_policy.per_domain_minimum` 块规定每个域最少多少不同 URL (耳机任务的代表性切分为 shopping 30、reddit 20、wikipedia 15)。
2. **引文总下限。**每任务在 Markdown 内至少 60 个引文 (`citation_policy.min_distinct_sources` 与 `markdown_spec.min_citations`), 且黄金池中的 `must_cite_urls` 至少 120 条。120 这一数字是「可见性下限」: 它强制智能体真正去做跨源检索并寻找独立证据, 而非吐出 Tavily 的前三条结果。
3. **冻结的黄金池。**每个任务有一份 `data/golden/deep/dr_cross_deep_XXXX.json`, 由 `scripts/build_deep_golden.py` 一次性从干净沙盒快照中爬取。文件含 `must_cite_urls` 列表 (智能体应当命中的 URL, 带权重)、`expected_pool_urls` 列表 (称职智能体应当找到的更广泛语料)、`triples` 列表 (主-谓-宾-source_url-quoted_span 五元组, 驱动 `claim_nli`)。两份池在快照上一次性爬取, 从不重生成, 使得 `recall` 的分母被冻结。

一个代表性任务是 `dr_cross_deep_0001` (消费级耳机市场情报), 意图 31 句, 要求:

- A 节 (产品图景): 至少 40 个不同商品页, 跨至少 6 个品牌、3 个价位区间, 每件商品给出价格/星级/评论数/特征论断。
- B 节 (社区情绪): 至少 30 个 Postmill 帖, 跨至少 4 个子版, 分为 5 类情绪。
- C 节 (技术依据): 至少 25 篇维基文章支撑 A 节的技术论断, 9 个指定文章 (Active noise control、Noise-cancelling headphones、Bluetooth、AptX、LDAC、Headphones、Loudspeaker、Lithium-ion battery、Wireless microphone) 强制覆盖。
- D 节 (跨源综合): 至少 5 处商品特征与维基的矛盾、按 Reddit 情绪给品牌排名、至少 3 处「评分 vs 情绪」分歧, 以及一份 top-10 购买建议, 每条至少 1 个 shopping URL、2 个 reddit URL、1 个 wiki URL 的证据。

完整意图 4144 字符, 该任务的黄金池为 `must=125`, `pool=739` (记录于 `spec` 的 `author_notes`)。该任务是三个「推荐锚点」(0001 / 0002 / 0005) 之一, 用于测试智能体能否给出排序型交付物; 其余任务覆盖比较、辟谣、因果分析、时间线、枚举等模式。

每任务 checklist 21 项, 均人工撰写, 集中存于 `data/tasks/deep_research/cross_site_deep/che`

3 13 个开源 DR 框架适配

3.1 适配范式

每个框架最终都跑在同一份 `shim` 与同一份 `ds_proxy` 之上。适配层分为三类:

1. **进程内 (in-process)**。在 `scripts/run_deep_task.py` 中定义一个函数, 直接 `import` 框架并在 `.venv-camel` 内运行。常见于 Python 主流 SDK 暴露良好的框架 (`camel-ai`、`smolagents`、`langchain-odr`)。
2. **子进程 (subprocess)**。写一份 `driver` 脚本到 `/tmp/`, 以另一份 Python (该框架的虚拟环境) 启动。适用于依赖冲突严重 (例如 `gpt-researcher` 依赖 `pre-1.0` 的 `LangChain` 子模块) 或必须使用其自身配置机制的框架。
3. **清理后的运行器 (clean runner)**。`scripts/runners/` 下的自包含模块, 导出 `async def run(intent, model, shim_url, proxy_url) -> str`, 由 `scripts/runners/registry.py` 自动发现, 然后合并到 `run_deep_task.py:1336-1356` 的分派表中。

分派表 `RUNNERS = _build_runners_map()` 位于 `scripts/run_deep_task.py:1359`。

3.2 camel-ai (in-process)

`camel-ai` 是单循环 `ChatAgent` + 显式工具调用结构。集成 (`scripts/run_deep_task.py:345-413`) `monkey-patch tavily.TavilyClient.__init__`, 使下游创建的任何 `TavilyClient` 实例的 `base_url` 都被重定向至 `shim`; 随后用 `SearchToolkit().search_tavily` 作为唯一工具构造 `ChatAgent`, 通过 `ModelFactory.create(model_platform=OPENAI_COMPATIBLE_MODEL, url=proxy)` 接入模型, 然后跑一次 `agent.step(intent)`。系统提示词手写, 含「NEVER invent URLs」与「至少引用 15 篇 Wikipedia 文章」两条, 因为 `camel` 默认不会跨三个语料分配查询。后处理一段会剥离偶发的思维链前缀 (「I now have enough data to compile...」)。虚拟环境 `.venv-camel`, 入口 `scripts/run_deep_task.py:345`。

3.3 smolagents (in-process)

smolagents 是 HuggingFace 的轻量工具调用循环集成 (scripts/run_deep_task.py:288-342) 使用 ToolCallingAgent 而非 CodeAgent, 因为 CodeAgent 生成的 Python 代码会用字符串模板构造 URL, 早期实验中约 92% 是捏造 URL。ToolCallingAgent 强制模型输出结构化 tool-call JSON, URL 必须从搜索结果原文 copy。Tavily 的 patch 与 camel-ai 同; LLM 走 OpenAIServerModel(api_base=proxy); 工具栈是 ApiWebSearchTool (指向 shim/search) + VisitWebpageTool。max_steps=60 给得足够大以保证迭代不被截断。虚拟环境 .venv-smol, 入口 scripts/run_deep_task.py:288。

3.4 gpt-researcher (subprocess via clean runner)

gpt-researcher 0.12.3 是「计划器 + 多并行 writer + 汇总器」框架。它 import 多个 pre-1.0 LangChain 子模块 (langchain.docstore、langchain.vectorstores 等), 这些子模块在 LangChain 1.x (本仓库主用 .venv-camel 上跑 langchain-odr 用的版本) 中已经不存在。gpt-researcher 因此需要独占虚拟环境 .venv-gptr。集成 (scripts/runners/gpt_researcher_runner.py) 写 driver 到 /tmp/ 并以 .venv-gptr/bin/python 启动。driver monkey-patch gpt_researcher.retrievers.tavily.tavily_search 把 self.base_url 设为 f"{shim_url}/search"; 设置 OPENAI_BASE_URL 指向代理; 使用 custom:text-embedding-v4 别名以保证 embedding 调用也走代理 (原 openai:embedding-3 别名会绕过 OPENAI_BASE_URL 直击真 OpenAI, 详见 runner 源码 scripts/runners/gpt_researcher_runner.py:17-19)。定义其排行榜位置的核心坑是 slug 模板 bug: 它通过把商品名 kebab-case 化为 /products/<slug> 来构造 Magento URL, 但 Magento 真实 slug 含智能体看不到的数字后缀。可达性几近为零 (composite_v2 均值 = 0.013)。

3.5 langchain-open_deep_research (in-process)

LangChain ODR 是基于 langgraph 的「supervisor → researcher → writer」流水线。集成 (scripts/run_deep_task.py:674-738) 同时 patch tavily.TavilyClient (同步) 与 tavily.AsyncTavilyClient (异步), 因为原来的 patch 只抓到了同步客户端而 ODR 用的是异步。异步 patch 把 self._api_base_url 设为 shim, 并冗余地在 init 之后覆盖 self._client.base_url。图被编译并 ainvoke, config 把六个模型通道 (research_model、compression_model、final_report_model、summarization_model、writer_model、planner_model) 全部设为 openai:<model>, 搜索 API 设为 tavily, 迭代上限调小 (max_researcher_iterations=5、max_react_tool_calls=8) 以约束墙钟。失败模式是其检索缓存 miss 时回退到输出外部 URL (真实的 en.wikipedia.org、真实的 reddit.com) 而非沙盒 URL。composite_v2 均值 = 0.001。虚拟环境 .venv-langchain-odr, 入口 scripts/run_deep_task.py:674。

3.6 ii-researcher (subprocess)

ii-researcher (Intelligent-Internet) 是 ReAct 式的「搜索链」智能体。集成 (scripts/run_deep_task.py:1098-1208) 写一份 driver,patch 子进程中的 tavily.TavilyClient 把 base_url 设为 shim, 强制 SEARCH_PROVIDER=tavily 与 SCRAPER_PROVIDER=bs (BeautifulSoup, 使请求走我们 patch 过的 requests.Session.send), 清掉所有代理环境变量, 并装上 HTTP 层拦截 preamble (_build_intercept_preamble,scripts/run_ 把 en.wikipedia.org/wiki/<X> 形 URL 在传输层重写为 localhost:8090/content/wikipedia_en_a ii-researcher 的搜索链通常在 3 次左右工具调用之后就终止, 不管意图多长, 因此一份报告往往只剩 5 到 15 条引文, 远低于 60 的下限。composite_v2 均值 = 0.001。虚拟环境 .venv-ii, 入口 scripts/run_deep_task.py:1098。

3.7 dzhng (subprocess via Node HTTP API)

dzhng 是 Node/TypeScript 的深度研究框架, 对外暴露 http://localhost:3051/api/generate-r Python 集成 (scripts/run_deep_task.py:1238-1257) 因此极小:POST {query, breadth, depth} 然后等待最多 1800 秒。Node 侧把 OpenAI 客户端与 Firecrawl 客户端分别指向 ds_proxy 与一个 Firecrawl 兼容适配器。集成踩坑是 API key 传递:Node 侧从自己的 .env 读 OPENAI_API_KEY 而非父 shell, 因此该值要同时设在 Node 进程环境与代理处。dzhng 当前数据集下有效打分不足 30, 未进入头部排名。

3.8 flowsearcher-ds (in-process, 自研)

flowsearcher-ds 是本仓库自研的 DR 智能体, 核心是记忆模块 src/memory/hierarchical.py 的三层记忆 (L1 任务级、L2 意图级、L3 全局)。它使用「记忆引导的规划」把搜索偏向于历史上 (本智能体 + 同意图过往运行) 发现的有效子查询, 并在发出工具调用前先输出结构化计划。入口是 scripts/run_flowsearcher.py 的 run_flowsearcher(intent, model, task_id) (scripts/run_deep_task.py:1231-1235)。集成是进程内, 因为它是本仓库的一部分, 跑在 .venv-camel 中。flowsearcher-ds 在排行榜上的位置 (Elo 1401, 第 2 名) 反映了记忆引导规划的价值:30 份打分均值 composite_v2 = 0.296, 低于 camel-ai 的 0.410, 但尾部更窄 (近 0 的运行更少), 这正是成对 Bradley-Terry 所奖励的。

3.9 deerflow (subprocess via clean runner)

DeerFlow 是 langgraph + Tavily 的多智能体框架。集成 (scripts/runners/deerflow_runner.py) 在 DeerFlow 自有根目录写 conf.yaml 与 driver, 设置 BASIC_MODEL__base_url / BASIC_MODEL__model / BASIC_MODEL__api_key 环境变量 (DeerFlow 官方的 LLM 配置面), 并在 DeerFlow 包装层 import 之前设置 langchain_tavily._utilities.TAVILY_API_URL 为 shim。driver 还把 DeerFlow 默认基于 Jina 的爬虫替换为调用 shim /extract 的轻

量版本, 因为 Jina 默认 POST 到 <https://r.jina.ai/>, 沙盒内不可达。driver 又强制 `aiohttp.ClientSession(trust_env=False)`, 以防 WSL 的 `/etc/environment` 代理变量泄漏; 这是修复「真 Tavily API 拒绝伪造 key」失败模式的关键改动。`composite_v2` 均值 = 0.023。虚拟环境 `third_party/deer-flow-v1/.venv`。

3.10 ldr (subprocess via clean runner)

LDR (LearningCircuit local-deep-research) 走自家程序化 API (`detailed_research()` 配合 `create_settings_snapshot`)。集成 (`scripts/runners/ldr_runner.py`) 写子进程 driver, 通过 LDR 官方的 `overrides=` 参数传 settings snapshot, 不 monkey-patch LDR 内部。driver 装上 HTTP 层拦截, 把 `api.tavily.com` 重写为 shim, 并在 LLM API 调用中把 `localhost:NNNN` 掩码为伪主机名 (`onestopmarket.com`、`postmill.net`、`kiwipedia.org`、`searchapi.internal`), 以避免 DeepSeek V4-flash 安全过滤器在请求体含 `localhost:NNNN` 时直接拒答 (一个早期踩坑, 见 `scripts/runners/ldr_runner.py:72-80`)。LDR 在本基准上的失败模式是把 Kiwix 路径转小写;Kiwix 对文章名大小写敏感, 因此维基域可达性静默归零。`composite_v2` 均值 = 0.029。虚拟环境 `.venv-ldr312`。

3.11 qx-agents (subprocess via clean runner)

qx-agents (qx-labs/agents-deep-research) 构建于 openai-agents SDK, 有 3 个可配置的模型通道与一套 SearchXNG 搜索适配器。集成 (`scripts/runners/qx_runner.py`) 把 `SEARCH_PROVIDER=searchxng`, 把 `SEARCHXNG_HOST` 指向本地 SerperAdapter (`scripts/runners/serper_adapter.py`), 后者把 SearchXNG GET 线协议翻译为 Tavily shim POST, 且在 import 任何 agent 模块之前先设 `agents.run_config.DEFAULT_MAX_TURNS = 30`。qx-agents 的失败模式是结构性而非配置性: 在 DeepSeek V4-flash 下, SDK 抛出 `pydantic.ValidationError: 2 validation errors for KnowledgeGapOutput` 与下游 `IndexError: list index out of range`, 每个任务都如此。所有 30 份运行都被退化样本过滤器 (`scripts/build_deep_leaderboard.py:60-79`) 剔除, qx-agents 从 Elo 表中被排除, 但在审计行里被显式记录。虚拟环境 `.venv-qx`。

3.12 tongyi-dr (subprocess via clean runner)

Tongyi DeepResearch 是阿里 NLP 的 ReAct 循环, 基于 `qwen_agent`。Tongyi 30B-A3B MoE checkpoint 在 bf16 下大约需要 60 GB 显存; 评测主机 RTX 5090 只有 32 GB。集成 (`scripts/runners/tongyi_runner.py`) 因此写一份 driver, 把 Tongyi 的 OpenAI 客户端重定向至 `ds_proxy` (LLM 是 DeepSeek V4-flash, 而非 Qwen)。Tongyi 五件工具保留: Search (替换为 shim 兼容 POST)、Visit (替换为 `requests.get` + LLM 摘要)、Scholar

(返回空, 沙盒无 scholar)、Python (返回错误, 沙盒无 SandboxFusion)、FileParser (禁用)。MAX_LLM_CALLS=50 上限总迭代。Tongyi 未进入头部排名。虚拟环境 `.venv-tongyi`。

3.13 deepagents (subprocess via clean runner)

LangChain DeepAgents 是带规划 (`write_todos`)、子智能体生成、文件系统工具的 langgraph 超级智能体。DeepAgents 不自带搜索工具; 集成 (`scripts/runners/deepagents_runner.py`) 定义一个薄薄的 `internet_search()` 函数, POST 到 Tavily shim, 然后作为工具传给 `create_deep_agent(tools=[...])`。LLM 通过 `init_chat_model("openai:<model>", base_url=proxy, api_key=...)` 接入。输出从 `result["messages"][-1].content` 取。DeepAgents 因打分样本数不足而未进入头部排名。虚拟环境 `.venv-camel` (共用) 或 `.venv-deepagents`。

3.14 storm + co-storm (subprocess via clean runner)

STORM 与 Co-STORM 是 Stanford 的 `knowledge_storm` 包。STORM 集成 (`scripts/runners/storm_runner.py`) 实现一个 `SandboxSearchRM`, 继承 `dspy.Retrieve` 并向 shim 发搜索, 然后驱动标准 STORM 流水线 (`run_knowledge_curation_module` → `run_outline_generation_module` → `run_article_generation_module` → `run_article_polishing_module(do_polish_article=True)`)。基准后期才发现并定位的关键 bug 是: runner 只读 `storm_gen_article_polished.txt` (文章正文), 从不把 `url_to_info.json` 这份独立的参考文献映射重新合并回 Markdown。智能体内部其实正确引用了, 但我们打分的是被剥离引文的正文。修复 (`scripts/runners/storm_runner.py:299-313`) 在尾部追加一段从 `url_to_info.json` 重建的 `## References` 节。修复之后, STORM 的 `composite_v2` 从「每次都是 0.000」变为修复后小批量 re-run 上的 ≈ 0.067 , 是本基准中单次修复带来的最大增量。Co-STORM (`scripts/runners/costorm_runner.py`) 共享同一份 `SandboxSearchRM`, 但还把 `knowledge_storm` 的 LiteLLM 风格 Encoder 替换为本地 `SentenceTransformer` (`paraphrase-MiniLM-L6-v2`), 因为 DeepSeek 不暴露 embedding 端点。STORM 在头部排名中位列第 9; Co-STORM 集成完成但打分样本不足。

3.15 框架汇总

公共主线是四件事: (a) 把 Tavily 客户端指向 shim; (b) 把 OpenAI 客户端指向 `ds_proxy`; (c) 拦截任何漏过 (a) 的 `en.wikipedia.org` 或 `api.tavily.com` URL; (d) 对大小写敏感语料, 不要把 URL 路径归一化。

表 2: 13 个框架的接入方式

框架	类型	接入要点	虚拟环境
camel-ai	单循环 ChatAgent	Tavily monkey-patch + ds_proxy 环境变量	.venv-camel
smolagents	ToolCallingAgent	Tavily patch + ApiWebSearchTool	.venv-smol
gpt-researcher	计划器 + 并行 writer	子进程 driver + custom embedding	.venv-gptr
langchain-odr	langgraph 三角	Tavily 同步 + 异步双 patch	.venv-langch
ii-researcher	ReAct 搜索链	子进程 + 拦截 preamble	.venv-ii
dzhng	Node HTTP API	本机 3051 POST	Node + .venv
flowsearcher-ds	记忆引导规划	进程内, 本仓库自研	.venv-camel
deerflow	langgraph 多智能体	conf.yaml + 环境变量 + Jina 替换	deer-flow-v1
ldr	程序化 detailed_research	settings_snapshot + 掩码	.venv-ldr312
qx-agents	openai-agents SDK	SearchXNG → Serper 适配器	.venv-qx
tongyi-dr	qwen_agent ReAct	重定向 LLM, 工具重写	.venv-tongyi
deepagents	langgraph 超级智能体	自定义搜索工具 + init_chat_model	.venv-deepag
storm + co-storm	knowledge_storm 流水线	SandboxSearchRM + 参考文献合并	.venv-storm

4 评分体系设计

4.1 七维评分概览

每份智能体报告在 7 个维度上打分。前三个是确定性的真实性探针 (URL 覆盖、URL 可达、引文文本匹配), 其后三个是带确定性守卫的 LLM 评分细则 (checklist、引文对齐、表达), 第七个 (分析深度) 是结构性检查与 LLM 细则的混合。复合分数 v1/v2 只用前四个维度, v3 用上全部七个维度加上表达与分析深度。

表 3: 七个评分维度概览

维度	取值	类型	耗时	进入哪个复合
url_coverage	[0,1]	确定性	毫秒	v1, v2, v3
url_reachability	[0,1]	HTTP 探活	秒	v1, v2 (硬门), v3 (floored)
quote_match	[0,1]	确定性	秒	v3 (可选 v2)
claim_nli	[0,1]	LLM 判官	分钟	头部 v2 中已弃
checklist	[0,1]	LLM 判官 + 守卫	秒	v1, v2, v3
citation_alignment	[0,1]	LLM 判官 (每引文)	分钟	v3
presentation	[0,1]	确定性 + 判官	秒	v3
analysis_depth	[0,1]	确定性 + 判官	秒	v3

4.2 url_coverage

URL 覆盖度量「智能体是否引用了它应当引用的 URL」。三个子分:

$$\begin{aligned} \text{must_cite_recall} &= \frac{\sum_{u \in \text{cited} \cap \text{must}} w_u}{\sum_{u \in \text{must}} w_u} \\ \text{pool_coverage} &= \frac{|\text{cited} \cap \text{expected_pool}|}{|\text{expected_pool}|} \\ \text{domain_balance} &= \min_{d \in \{\text{shop}, \text{reddit}, \text{wiki}\}} \min \left(1, \frac{\text{cited}_d}{\min_d} \right) \end{aligned}$$

得分是加权和 (默认权重 0.55 must-cite、0.15 pool、0.30 domain)。维度通过条件: $\text{must_cite_recall} \geq 0.45, \text{pool_coverage} \geq 0.12, \text{domain_balance} \geq 0.80, \text{unique cited URL 数} \geq 60$ (`src/verifiers/url_coverage_verifier.py:166-186`)。三项并存的原因: must-cite 单独奖励命中冻结池的智能体, 但会惩罚找到同等质量替代页面的智能体; pool_coverage 补偿这一点; domain_balance 是「过引一个语料、丢弃另一个」的唯一信号。URL 在比较前由 `canonicalize_url` (`src/verifiers/citation_format.py`) 规范化 (剥尾斜杠、域名转小写、丢 fragment、query 排序)。

4.3 url_reachability

可达性是抵抗 URL 捏造最强的单一信号。验证器从 Markdown 中抽出每一个被引用的 URL, 过滤至沙盒 `host:port` 严格匹配 (而非子串, 避免 `localhost:7770` 匹配 `localhost:77703`), 对每个 URL 发起 6 秒超时的 GET, 3 次重试, 指数退避, 然后数 200 响应 (`src/verifiers/url_reachability_verifier.py:55-82`)。

$$\text{reachability} = \frac{|\{u : \text{status}(u) = 200\}|}{|\{u : \text{status}(u) \neq 5xx \wedge \text{status}(u) \neq \text{net_fail}\}|}$$

分母是**可解析**的探针数 (不是总数), 因此 30% 5xx 或网络失败不会被算作智能体捏造 URL; 相反, 验证器会在「全部探针 5xx/net-fail」时把 `infra_failure` 置为 True, 在「至少 30% 探针 5xx/net-fail」时把 `infra_warning` 置为 True。这一拆分是排行榜把「智能体捏造 URL」(高 4xx) 与「沙盒当时挂了」(高 5xx) 分开的关键。维度通过条件: $\text{reachability} \geq 0.30$ 。

4.4 quote_match

引文文本匹配抓的是「URL 存在但页面并不包含支撑论断的内容」。验证器从 Markdown 抽取 (`url`, `claim_context`) 对 (`claim_context` 是引文位置周围 200 字符的窗口, Markdown 链接语法被替换为可见 label), 用同一份重试-退避的 `_fetch` 助手抓取每一个唯一 URL, 并计算 claim token 与页面 token 的重叠率:

$$\text{overlap} = \frac{|\text{tokens}(\text{claim}) \cap \text{tokens}(\text{page})|}{|\text{tokens}(\text{claim})|} \geq \theta$$

阈值 $\theta = 0.10$ (src/verifiers/quote_match_verifier.py:91-95), 通过的对数除以总对数即为本维度得分。验证器对 claim token 做 stop-word 过滤, 避免「the、of、and」抬高分母。这一维度是设计上最便宜的: 无 LLM 调用, 无 NLI 判断, 只是 BM25 风格的重叠。

4.5 claim_nli

claim_nli 是 quote_match 的「重型版本」: 对每一个在黄金中带有 quoted_span 的 (claim_context, cited_url) 对, 验证器经 ds_proxy 询问 DeepSeek V4-flash:

```
CLAIM: ... SOURCE_QUOTE: ... Does the source quote DIRECTLY support
the claim? Output JSON {entail, prob, reason}.
```

在零温度下, 验证器认为当 prob >= 0.80 (默认 θ , 与 ReClaim [14] 一致) 时论断被支撑; 最终得分为通过比例, 每份报告 max_calls=80 上限以约束成本。自 **2026-04-27** 起, claim_nli 从头部 composite_v2 中被剔除 (乘数置 1.0), 原因是 DeepSeek-V4-flash 的 NLI 判官在视觉噪声大的商品页上产生过多假阴性。该维度仍然每次运行都被计算与持久化以保证可追溯。每份报告该维度耗时 30 到 120 秒。

4.6 checklist

checklist 是 21 项每任务专属的二值细则。每任务的条目人工撰写, 集中存于 data/tasks/deep_research/cross_site_deep/checklists_deep.json。判官是经 ds_proxy 调用的零温度 DeepSeek V4-flash, 系统提示要求逐行输出 PASS/FAIL/UNCLEAR。三层确定性守卫包住 LLM 判官:

1. **退化答案守卫** (scripts/score_deep_answer.py:111-123): 若答案为空或 50 词以内且无引文, 所有项不调 LLM 直接 FAIL。这条捕获 (qx-agents error: ValidationError ...) 这类被判官误判为 21/21 PASS 的情况。
2. **全 PASS 短答案守卫** (scripts/score_deep_answer.py:165-170): 若判官在所有项上都返回 PASS 但答案少于 500 词, 所有判决降级为 FAIL。这条捕获 DeepSeek V4-flash 在短输入上记录在案的「松懈模式」bug。
3. **锚定关键词交叉检查** (scripts/score_deep_answer.py:174-180): 若 URL 可达性 < 0.30, 则任何含锚定关键词 (「URL」「cited」「linked」「sandbox」「domain」「reddit」「wikipedia」「shopping」「thread」「article」「page」「forum」) 的项被降级为 FAIL。正是这一守卫阻止了 gpt-researcher 在引了 93 个伪造 URL 的报告上拿到 21/21 PASS (任务 0009 审计案例)。

得分为 `pass_count / 21`。

4.7 citation_alignment

引文对齐是 ALCE 风格的每引文 NLI 探针。对每一个 (`sentence`, `cited_url`) 对, 验证器先抓取被引页面, 剥 HTML (移除 `script`、`style`、`nav`、`footer`、`header`、`noscript`、`aside`、`svg`), 截断到 8000 字符, 询问判官:

```
Does the page SUPPORT the claim? Output exactly "VERDICT: SUPPORTED"
or "VERDICT: NOT_SUPPORTED"
```

然后计算:

$$\text{precision} = \frac{\text{supported pairs}}{\text{total pairs}}, \quad \text{recall} = \frac{\text{sentences with } \geq 1 \text{ supported cite}}{\text{sentences with any cite}}$$

得分采用 `precision` (`src/verifiers/citation_alignment_verifier.py:321`)。资源上限: 每份报告 200 对、每页正文 8000 字符、4 worker 线程池抓取、每对单次判官调用 `max_tokens=80`。验证器同时识别 `[label](url)` 与裸 `https://...` 两种引文形态 (裸 URL 形态正是 `ldr` 触发 `quote_match` 返回 `claims_total=0` 的原因)。

4.8 analysis_depth 与 presentation

这两个维度衡量「报告作为研究交付物读起来如何」, 而非「引文如何」。两者都是「确定性结构检查 + LLM 评分细则」的二层混合。

analysis_depth 共 10 项 (4 确定性 + 6 LLM, 得分 = $0.3 \cdot \text{tier_a} + 0.7 \cdot \text{tier_b}$):

- Tier A: 至少一段含 ≥ 2 个域的 URL; ≥ 3 段使用「矛盾」语言; ≥ 3 段被 ≥ 3 个跨 ≥ 2 个域的 URL 支撑; 比较性语言密度 ≥ 0.15 。
- Tier B: 超越枚举、因果推理、信息源三角、新见解、反驳意识、可行动结论。

矛盾与比较性语言的正则见 `src/verifiers/analysis_depth_verifier.py:38-52`。域提取在 `host` 为 `localhost` 时使用 `host:port`, 以保证三个沙盒语料不会塌缩为一个域。

presentation 共 12 项 (6 确定性 + 6 LLM, 得分 = $0.4 \cdot \text{tier_a} + 0.6 \cdot \text{tier_b}$):

- Tier A: 标题层级 ≥ 3 ; ≥ 5 个 h2 节; 节平衡 $CV < 1.5$; 至少一份列表或表格; 无 200 词以上无标题孤立段; Flesch Reading Ease ≥ 30 。
- Tier B: 逻辑流、过渡质量、无冗余叙述废话、格式一致、结论综合、专业语气。

Tier B 提示词对长报告会送「头 + 尾」而非只送头, 因为 `conclusion_synthesizes` 与 `consistent_formatting` 需要看到报告结尾 (`src/verifiers/presentation_verifier.py:217-22`)

这两个维度每次运行都会被计算并写入 `.score.json`, 但只对 `composite_v3` 有贡献。头部 `composite_v2` 主动不纳入它们: 如果把「专业语气」与「URL 可达性」放到同一根坐标轴上, 真实性门反转 (finding 6.4.1) 的现象就看不到了。

4.9 复合分数 `composite_v1 / v2 / v3` 的推导

三个复合分数共存于 `src/scoring/leaderboard_composites.py`, 三者共享同一份「质量项」 Q , 区别仅在于如何用可达性门控质量。质量项为:

$$Q = 0.40 \cdot \text{url_coverage} + 0.40 \cdot \text{checklist} + 0.20 \cdot \text{spec} \quad (1)$$

其中 `spec` 是 Markdown 规范标志的通过率 (`words_ok`、`citations_ok`、`paragraphs_ok`; 见 `spec_pass_fraction,src/scoring/leaderboard_composites.py:28-35`)。

Composite v1 (legacy 加性)。可达性被折叠进质量输入, 不充当门:

$$\text{composite}_{v1} = Q \quad (2)$$

(`composite_v1` 于 `src/scoring/leaderboard_composites.py:60-68`)。这是「无真实性门」基线, 在排行榜中只用于演示 `v2` 反转 `v1` 的排名。

Composite v2 truthful (乘性, 头部)。可达性作为乘性门:

$$\text{composite}_{v2} = \text{reachability} \cdot Q \quad (3)$$

(`composite_v2_truthful` 于 `src/scoring/leaderboard_composites.py:71-81`)。若 `reachability = 0`, 复合分 = 0。这是头部排名。此外, 可选的真实性分解变体

$$\text{composite}_{v2,\text{factored}} = \text{reach} \cdot (0.5 + 0.5 \cdot \text{qm}) \cdot (0.5 + 0.5 \cdot \text{nli}) \cdot Q \quad (4)$$

仍由 `score_deep_answer.py:233-243` 计算并持久化, 但头部数字采用朴素乘性形态 (3), 原因有二:(a) 2026-04-27 起 `claim_nli` 从头部弃用;(b) 排行榜构造器读取的是 `composite_v2_truthful` 字段, 而非按运行分解的 `factored` 字段。

Composite v3 (松弛门, 7 维)。可达性变为带下界的门, 且四个额外维度进入质量原始分:

$$\begin{aligned} \text{composite}_{v3} = \max(0.1, \text{reach}) \cdot (& 0.20 \cdot \text{url_cov} + 0.20 \cdot \text{qm} + 0.20 \cdot \text{checklist} \\ & + 0.10 \cdot \text{spec} + 0.15 \cdot \text{cit_align} + 0.10 \cdot \text{depth} + 0.05 \cdot \text{pres}) \end{aligned} \quad (5)$$

(`composite_v3` 于 `src/scoring/leaderboard_composites.py:107-127;scripts/score_deep_a` 中公式相同)。0.1 的下界使「只会捏造的智能体」不会拿到字面上的 0, 从而 `v3` 是比 `v2` 更柔和的排名信号。`v3` 每次运行都被算并持久化, 但头部排行榜仍用 `v2`。

为何 `v2` 是头部。乘性门是「捏造不可妥协」最简单的表达。一个 `reachability = 0` 的智能体在事实上没有产出可用证据; 用 0.1 倍权重 (`v3`) 奖励它, 会低估这份报告在实践中的负效用。`v2` 的硬零是让发现 6.4.1 (真实性门反转) 浮现的公式。`v3` 存在的两个理由是:(a) 它让逐维度分析可以表明, 即便加入四个额外细则, 真实性门所形成的簇仍然存在;(b) 它让审计页可以排序「`v2` 上为 0 但报告非空」的智能体, 使它们不至于静默消失。头部数字是 `composite_v2_truthful`; `v1` 在排行榜中并列显示以呈现反转; `v3` 出现于运行级审计, 但不驱动 Elo。

5 排名方法学

5.1 Bradley-Terry 模型

排名聚合自每任务的 227 个成对对战 (`storm` 由于只在 17 个任务上有有效打分, 只贡献 136 对战)。对战由 `composite_v2` 合成: 对每个任务, 对每一对都产出非退化打分的智能体, 复合分高者赢得该对战、低者输; 若 $|\Delta\text{composite}| < 0.005$, 记为平 (`scripts/build_deep_leaderboard.py:241-244` 与 `src/scoring/arena.py:33-39`)。tie-epsilon 取得很紧: 在 `v2` 下, 许多智能体会塌缩到 `composite = 0` (可达门把它们杀死了), loose epsilon 会让这些智能体彼此平, 而非按处理顺序决出胜负。

Elo 模型与 Chatbot Arena [11] 同。对每次对战 (a, b) , 观察到结果 $s_a \in \{0, 0.5, 1\}$ 时,

$$E_a = \frac{1}{1 + 10^{(R_b - R_a)/400}}, \quad R'_a = R_a + K(s_a - E_a) \quad (6)$$

$K = 32$, 起始分 1000, 算法运行 20 轮, 每轮 shuffle 对战次序, 最后对 20 轮的最终评分取平均 (`src/scoring/arena.py:84-120`)。多轮平均是为了消除单轮 Elo 在小数据上对对战顺序的敏感性。

成对比较的 Bradley-Terry 似然 [12] 写作:

$$\mathcal{L}(\pi) = \prod_{(i,j)} \left(\frac{\pi_i}{\pi_i + \pi_j} \right)^{w_{ij}}, \quad R_i = 400 \log_{10} \pi_i + C \quad (7)$$

其中 w_{ij} 是 i 击败 j 的次数。Elo 多轮平均可视为上述似然 MLE 的迭代近似。

5.2 Bootstrap 置信区间

95% 置信区间用非参数 bootstrap (`compute_elo_with_ci`): 对全部对战做 $B = 1000$ 次有放回重采样, 每次重采样重新拟合 Elo, 最后取 2.5% 与 97.5% 分位数。本基准头部

排行榜 CI 半宽介于 ± 43 与 ± 96 Elo 点之间; 较窄者 (例如 storm 的 ± 43) 是参与对战数较少的智能体。

5.3 置换 p 值显著性检验

成对置换显著性 (`rank_significance_test`, $N = 1000$) 在相邻名次对内 shuffle 对战结果, 然后问: 在「两个智能体等强」的零假设下, 我们观察到至少这么大 Elo 差距的概率是多少?

5.4 退化样本过滤规则

将退化样本纳入 Bradley-Terry 会扭曲评分。runner-error placeholder (例如 (`qx-agents error: ValidationError ...`)) 是一个字符串, 打分器从中愉快地抽不出 URL, LLM 判官也愉快地在近乎空的答案上输出 PASS 或 FAIL。最终复合分 (≈ 0.02) 既不算赢任何人, 也不算输给任何人。过滤器在 Bradley-Terry 之前剔除它们 (`scripts/build_deep_leaderboard.py:82-131`)。四个模式被剔除:

1. 短答案 + 无信号。 `answer_chars < 600` 且 `url_reachability.score = 0` 且 `checklist.pass_rate = 0`。
2. 沙盒基础设施失败。 `url_reachability.details.infra_failure == True`。每次探针都是 5xx 或 net-fail, 意味着沙盒打分期间挂了, 不是智能体捏 URL。
3. 判官全失败。 `checklist.judge_error` 且 `analysis_depth.judge_error` 且 `presentation.judge_error`。三个 LLM 判官全错 (通常是 `ds_proxy` 超时), 运行没有判官信号。
4. Runner-failure / runner-exception 占位文本。答案 Markdown 以 (`<Agent> produced no report after Ns, exit=N`) 开头 (`_RUNNER_FAILURE_PREFIX_RE`) 或以 (`<Agent> error: ...`) / (`<Agent> stderr: ...`) 开头 (`_RUNNER_EXCEPTION_PREFIX_RE`)。

排行榜审计行专门保留了被过滤的智能体的元信息, 以防止它们「未被基准测试过」式地静默消失。

6 实验结果

6.1 实验设置

骨干模型: DeepSeek V4-flash 非推理模式 (`thinking: disabled` 由 `ds_proxy` 注入)。任务: 56 个 `dr_cross_deep_XXXX` 跨站点深度任务。智能体: 9 个进入头部排名 (camel-ai、flowsearcher-ds、smolagents、ldr、gpt-researcher、deerflow、ii-researcher、langchain-odr、

storm), 加上被审计行记录的 qx-agents、Tongyi-dr、dzhng、Co-STORM、deepagents 等若干个不进入头部的框架。score 文件总数:283。

每个运行的资源上限是 1800 秒墙钟、ds_proxy 与 shim 在沙盒侧无限调用 (实际由模型与框架的逻辑约束)。LLM 判官调用一律在零温度下进行, 每份报告每个 LLM 维度的调用次数有明确上限 (claim_nli max_calls=80、citation_alignment 每报告 200 对)。

6.2 最终排行榜 (composite_v2_truthful)

表 4: 头部排行榜 (composite_v2_truthful,56 任务,DeepSeek V4-flash)

#	智能体	Elo	95% CI	W	L	D	对战
1	camel-ai	1480.6	[1405, 1567] ± 81	206	20	1	227
2	flowsearcher-ds	1401.2	[1324, 1486] ± 81	193	32	2	227
3	smolagents	1307.1	[1211, 1404] ± 96	170	42	15	227
4	ldr	945.1	[871, 1014] ± 72	80	100	47	227
5	gpt-researcher	865.8	[793, 941] ± 74	60	114	53	227
6	deerflow	821.7	[756, 903] ± 74	40	118	69	227
7	ii-researcher	731.5	[676, 776] ± 50	5	130	92	227
8	langchain-odr	728.8	[674, 781] ± 53	6	129	92	227
9	storm	718.2	[677, 763] ± 43	0	75	61	136

排名表加上审计行的 4 个被排除/部分排除框架 (qx-agents、flowsearcher-ds 部分、storm 部分、Co-STORM/Tongyi/deepagents/dzhng 因样本不足), 覆盖 13 个适配框架。

6.3 相邻名次显著性

只有 smolagents 与 ldr 之间的 362-Elo 鸿沟在 $\alpha = 0.05$ 水准上显著。前三名是统计上并列的「头部簇」; 后六名是统计上并列的「尾部簇」。把这份排名读作两个簇被一条窄鸿沟隔开, 而非 9 个有序档次, 是更准确的解读。

逐维度 Elo。 LEADERBOARD_DEEP.md 中的 render_per_pillar_table 输出:

逐维度表是本基准最清晰的单一产物。注意 gpt-researcher 在 url_coverage 上 Elo 最高 (1305: 它确实输出很多看起来像 Magento URL 的字符串, 其中若干凑巧是黄金 URL), 在 checklist 上第二高 (1152:LLM 判官认为报告读着不错), 但在 reachability 上倒数第三 (883:URL 实际上解析不了)。composite_v2 的乘性门正是阻止这一画像登顶的因素。

表 5: 相邻名次置换 p 值 ($N = 1000$)

高名次	低名次	差距 (Elo)	p-value	显著 ($\alpha = 0.05$)
camel-ai	flowsearcher-ds	79.4	0.305	否
flowsearcher-ds	smolagents	94.1	0.447	否
smolagents	ldr	362.0	0.000	是
ldr	gpt-researcher	79.3	0.326	否
gpt-researcher	deerflow	44.1	0.573	否
deerflow	ii-researcher	90.2	0.310	否
ii-researcher	langchain-odr	2.7	0.970	否
langchain-odr	storm	10.6	0.896	否

表 6: 逐维度 Elo

智能体	checklist	quote	reach	spec	url_cov
camel-ai	1170	1486	1287	1268	1182
flowsearcher-ds	1147	1398	1256	1269	1141
smolagents	1088	1317	1236	1003	1146
ldr	614	789	1215	644	908
gpt-researcher	1152	798	883	1238	1305
deerflow	943	824	860	1057	820
ii-researcher	970	805	752	772	816
langchain-odr	1024	788	766	1098	932
storm	893	795	745	652	750

6.4 关键发现

6.4.1 真实性门反转朴素排名 (truth-gate inversion)

把同一份对战矩阵在 `composite_v1` (加性、无乘性门) 上重跑, 可以得到一张不同的排行榜:

三个智能体的名次变动达三档或以上。**gpt-researcher** 从第 5 跳到第 1: 其文笔流畅, checklist 分高, url_coverage 在九者之中第一; 一旦不再乘以可达性, 质量项就是全场最大。**langchain-odr** 从第 8 跳到第 5: 它的报告结构很好, 通过 checklist 与 spec, 只是 URL 是外部的。**ldr** 从第 4 落到第 9: 在门控之下它被「reach pillar 高分」(per-pillar reach Elo 1215, 高于 gpt-researcher) 救了一命; 一旦不再有门, 它就什么也不剩。

一个具体例子: gpt-researcher 在 `dr_cross_deep_0009` 上的报告 6200 词, 通过 21 项中的 19 项 checklist, presentation 0.78, 引用 93 个 URL; 其中只有不到 10 个能从沙盒返回 HTTP 200。该运行的 `composite_v1 = 0.387`, `composite_v2 = 0.000`。审计页

表 7: v2 vs v1 排名对比 (头号发现)

智能体	v2 Elo	v2 名次	v1 Elo	v1 名次	变动
gpt-researcher	866	5	1269	1	+4
flowsearcher-ds	1401	2	1253	2	0
camel-ai	1481	1	1232	3	-2
smolagents	1307	3	1088	4	-1
langchain-odr	729	8	1005	5	+3
deerflow	822	6	933	6	0
ii-researcher	732	7	898	7	0
storm	718	9	801	8	+1
ldr	945	4	520	9	-5

/audit 把这个例子标记为「流畅捏造」失败模式的标准样本。

支持「带门」公式的最强论点是: 一个读起来很美、引文却是捏造的 DR 智能体, 比一个文笔笨拙但引用真实页面的智能体**更糟糕**, 因为流畅捏造在下游被相信的概率更高。真实性门让这一点直接可见。

6.4.2 多智能体协作架构在本沙盒上占优

头部三框架中, 两个是多智能体协作 (camel-ai 的 ChatAgent + SearchToolkit、flowsearcher-ds 的记忆引导规划器 + 执行器), 一个是受约束的工具调用循环 (smolagents 的 ToolCallingAgent + max_steps=60)。底部六个中, 有两个是「不分解」的单循环或顺序流水线 (ii-researcher 的 ReAct 在 3 到 5 轮后终止; gpt-researcher 是「计划器 → 多 writer → 汇总」但每个 writer 是单次)。趋势与下面这条假设一致: DeepSeek V4-flash 作为非推理骨干, 得益于外部分解。当框架把任务分解为子查询并编排它们时, 模型能把每个子槽填好; 当框架要求模型在一个提示里隐式分解时, 输出要么截断要么漂移。

这个发现以沙盒为界: 120 引文下限正是强制分解的来源。DeepSeek V4-flash 的一轮前向几乎不可能在单次搜索之后产生 60 条跨源引文, 因此任何不在多轮搜索中重试的框架, 结构上就被卡在引文下限以下。换用推理型骨干 (DeepSeek V4 thinking on 或 GPT-5 reasoning) 时, 多智能体与单循环之间的差距可能收窄, 因为模型自身会替框架做更多分解。Web 应用 /compare 页面所规划的多 LLM 验证即此目的。

6.4.3 社区热度 vs 实测排名分歧

gpt-researcher 在本基准上排名第 5, 但其 GitHub 热度 25.7k stars, 是开源世界中「deep research」一词的事实参考实现。它的排名落后于一个大多数读者没听说过的框架 (flowsearcher-ds, 自研) 与一个并非专为 DR 而设的通用智能体循环 (smolagents)。

这一分歧有三层成因。

第一, 热度选择「产品级实现」, 而非「在固定评测上的实测质量」。gpt-researcher 是面向开放网的优秀工程: 其提示词针对真 Google/Tavily 调过, 其检索流水线假设真 CDN 提供 HTML, 其报告器期待开放网 URL 模式。把它放到一个 Magento 沙盒上 (URL slug 含数字后缀, 模型无法模板化), 它的工程优势不可迁移。

第二, 热度选择「文笔好」。大多数使用者读完报告就完事, 不会去点 93 个 URL 看有几个返回 200。gpt-researcher 的报告文笔好, 而以文笔评价 DR 智能体的社区做的事情, 正是 composite_v1 做的事。composite_v2 就是把差异显化的工具。

第三, 热度选择「容易上手」。gpt-researcher 的一行启动是真本事; smolagents 是没有任何 DR 营销光环的通用工具调用循环。易上手与「实测质量」相关性弱。

对这个领域的启示不是 gpt-researcher 不好, 而是「社区最爱」与「在密封沙盒上的实测质量」是两根不同的坐标轴, 而本基准的存在就是为了刻画第二根。

6.4.4 框架级失败模式

基准过程中捕获并 (部分) 修复的四个具体 bug:

- **qx-agents 框架级不兼容。**在 DeepSeek V4-flash 下, openai-agents SDK 抛 `pydantic.ValidationError: 2 validation errors for KnowledgeGapOutput` 与下游 `IndexError: list index out of range`, 每个任务都如此。不是本仓库该修的 bug。qx-agents 从 Elo 中被排除, 但在审计行公开记录。30 份打分文件全部由 `_looks_degenerate` 经 `_RUNNER_EXCEPTION_PREFIX_RE` (`scripts/build_deep_leaderboard.py:76-79`) 剔除。
- **storm 参考文献合并修复。**STORM runner 只读 `storm_gen_article_polished.txt` (正文), 从未把独立的 `url_to_info.json` URL 表合并回 Markdown。智能体内部其实正确引用了, 我们打分的是被剥引文的版本。修复 `scripts/runners/storm_runner.py:299-311` 在末尾追加一段从 `url_to_info.json` 重建的 `## References` 节后, STORM 的 `composite_v2` 从「每次都是 0.000」变为修复后小批量 re-run 上的 ≈ 0.067 。这是基准全程最大的单次修复增量。
- **Kiwix URL 别名 bug。**黄金池存了一种规范形态 (`localhost:8090/.../A/Page_Name`), 部分智能体输出另一种 Kiwix 前缀变体 (`/viewer#wikipedia/A/Page_Name`)。URL 规范化器 (`canonicalize_url` 于 `src/verifiers/citation_format.py`) 被打补丁以合并这两种形态; 64 份历史打分被重新打分, 3 对历史名次因此解锁。
- **dzhng API key 传递。**dzhng 的 Node 侧从自己的 `.env` 读 `OPENAI_API_KEY`, 而非父 shell。把 key 只设在启动 Node 的 bash 环境里不会传播。修复: 把 key 直接写到 `third_party/deep-research/.env`。

两个基础设施层面的事件:

- **运行中途 GPU 掉线。**评测主机 RTX 5090 在 `nvidia-smi: GPU is lost` 后失联; 基准并行迁移至 DeepSeek API, 无数据丢失地继续了 bulk run。
- **DeepSeek API 402 (余额不足)。**DeepSeek API 在后期一小段时间内返回 HTTP 402, 因预付余额清零。补充后继续。排行榜构造器的退化过滤器以「judge_error 三连命中」捕获了这段时间内完成的运行, 使其不污染 Elo。

7 讨论

7.1 工作的局限性

本基准在五个方向上具有局限性, 设计上是显式承认这些局限的。

三个语料, 而非整个网络。Magento + Postmill + Kiwix 覆盖了三种垂直模式 (电商、社区、百科), 但没有覆盖新闻、学术文献、原始文档, 以及登录墙背后的一切。某些在「金融报告合成」上表现优秀的智能体不会被本基准测出。代价是有意的: 能覆盖整个开放网的沙盒就是开放网, 而我们已经说明了为何那不工作。

头部数字使用单一骨干。头部排名中所有智能体都跑在 `deepseek-v4-flash` 非推理上。跨架构对比需要把同一批智能体放到多个骨干上; `/compare` 页与 `ds_proxy` 的可切换后端正是为此而设计, 但当前数字仍是单骨干。风险是: 用 DeepSeek 排出来的名次, 告诉我们的是「哪个智能体架构对 DeepSeek 最合适」, 而非「哪个智能体架构最通用」。

LLM-as-judge 偏差。七个维度中有三个 (`checklist`、`citation_alignment`、`presentation` 与 `analysis_depth` 的 LLM 部分) 都调用同一份 DeepSeek V4-flash 判官。确定性守卫 (退化答案跳过、全 PASS 短答案降级、锚定关键词交叉检查) 捕获了最坏的「自我偏好」与「松懈模式」失败, 但残余偏差非零。21 项 PASS/FAIL/UNCLEAR 比单一 Likert 难游戏化 (这是 DRACO [15] 风格的构造); 未来迭代应换用不同家族的判官 (GPT-5、Claude) 测量一致性。

部分任务覆盖。100 个设计任务中只有 57 个完成端到端打分; 剩余 43 个 spec 与黄金池就绪但未跑过。若全部 100 个都跑, 排行榜 Elo CI 半宽会从 $\pm 43 \sim \pm 96$ 收紧到大约 $\pm 30 \sim \pm 60$ 。

进程内 runner 虚拟环境冲突。三个进程内 runner (`camel-ai`、`smolagents`、`flowsearcher-ds`、`deepagents` 部分) 共用 `.venv-camel`。任一框架的依赖升级可能会掩盖另一框架的依赖冲突。子进程 runner 设计上每个独占一份虚拟环境, 因此避开了该问题, 但进程内 runner 做不到。这是「进程内集成」的已知代价; 未来一项清理是把每个框架都收进自己的虚拟环境并用 `clean runner` 启动。

第六个 (较小的) 局限是: 头部 `composite_v2` 自 2026-04-27 起弃用 `claim_nli` (乘数为 1.0), 原因是 NLI 判官在视觉噪声大的商品页上不可靠。原则上的修法是换非 DeepSeek 的 NLI 判官或手工校准; 两者都是未来工作。

7.2 工程实现要点

Web 应用实时读盘。Web 应用 `web/server.py` 在每次请求时从磁盘读取排行榜 JSON, 无数据库、无 Redis、无后台刷新。这一「实时读盘」模式 (`web/server.py:59-70`) 是中心化的简化: `bulk runner` 写下新的 `<agent>__<task>_matrix.score.json`, 等下一次 `build_deep_leaderboard.py` 跑完, 接下来对 `/api/leaderboard.json` 的调用就自动看到新结果。对于一个排行榜「分钟级」刷新的小基准, 这就够用了。

多虚拟环境隔离。13 个框架对应 ≥ 9 个虚拟环境 + 1 个 Node 运行时。任何一个框架的依赖升级不影响其余。这是子进程范式的核心优势: 进程内 patch 看似省事, 但任一升级都可能炸裂兄弟框架。

子进程 runner 框架隔离。清理过的 runner 提供统一接口 `async def run(intent, model, shim_url, proxy_url) -> str`, 通过 `registry.py` 自动发现, 合并到 `RUNNERS` 分派表。每个 runner 内部用自己框架的官方配置面 (DeerFlow 的 `conf.yaml`、LDR 的 `overrides=`、`qx-agents` 的 `SearchXNG`)。这一规约的回报是: 接入第 14 个框架, 所需要的是写一个 runner 模块并配一个虚拟环境, 无需改任何其它框架的代码。

7.3 公平性审计

`data/results/audit/` 下的最终审计报告 (`HUMAN_URL_AUDIT.md` 与 `final_audit.json`) 给出了一项独立的事后核验: 在 14 个被接入的框架中, 审计员人工抽检每个框架的代表性运行, 核对「智能体被打分文件里的 URL 是否真的出现在该智能体的报告 markdown 中」与「报告 markdown 是否就是该智能体真正的产物 (而非中间体或被截断)」。审计结论是: 14 个框架中 0 个存在我们这一侧的失误 (Our-Fault), 意即: 本基准对智能体的失败归因均成立。所有归零的运行, 都是智能体侧 (要么捏造 URL、要么超时、要么框架结构性 bug), 不是我们打分链路的失误。

8 总结与展望

8.1 工作总结

本论文构建了 Deep-Research Arena, 一套面向开源 DR 智能体的密封沙盒基准。三个本地化语料 (Magento、Postmill、Kiwix) + 两个粘合服务 (Tavily 兼容 shim 与 OpenAI 兼容 `ds_proxy`), 让 13 个开源 DR 框架在同一基底上、同一骨干上、同一打分栈上接受评测。九个框架在 56 个跨站点深度任务上产出有效运行, 合计 283 份打分文件。

打分体系由 7 个验证维度组成: 三个确定性真实性探针 + 四个 LLM 判官细则。三个复合分数共存: `composite_v1` 加性基线、`composite_v2_truthful` 乘性真实性门 (头部)、`composite_v3` 七维松弛门。基于 `composite_v2` 合成的成对对战, 经 Bradley-Terry/Elo 拟合 + Bootstrap 95% CI + 置换 p 值, 得到头部 9 行排行榜。

最重要的发现是真实性门反转: 在 v1 下首位的 `gpt-researcher` (Elo 1269) 在 v2 下降到第 5 (Elo 866); `camel-ai` 从 v1 第 3 升到 v2 首位 (Elo 1481)。流畅写作不是有据可查, 合并打分会奖励错误行为, 分离两者是本基准在方法学上的核心贡献。

附带发现包括: 多智能体协作架构在本沙盒上占优于单循环 (与非推理骨干在外部分解下表现更好这一假设一致); 社区热度 (GitHub stars) 与实测排名相关性弱; `storm` 的参考文献合并 bug 修复是本基准最大的单次增量 (`composite_v2` 从 0 升至 ≈ 0.067); `qx-agents` 因 SDK 在 DeepSeek 下抛 `pydantic ValidationError` 而结构性不可用。

8.2 未来研究方向

按优先级排序如下:

多 LLM 比较矩阵。最有结构性影响的下一步: 同样的智能体、同样的任务、同样的打分, 把 `ds_proxy` 后端从 DeepSeek V4-flash 换为 GPT-5、Claude Opus 4.7、Gemini、Qwen3.5 (推理开/关各一个变体), 产出 13×5 (智能体 $\times$ 骨干) 的 Elo 矩阵。`/compare` 页面已经布好路由, 瓶颈是 API 预算 (一轮全骨干约 2000 调用 $\times \$0.005 =$ 约 \$10 (DeepSeek), 五个骨干按更贵的算约 \$300/轮)。

更多垂直域沙盒。加一份新闻语料 (Common Crawl 切片) 与一份学术语料 (S2ORC 切片) 会让基准能就「时序推理」与「学术综合」给出独立的发现。`shim` 的索引接口是语料无关的, 只需要新增搜索索引插件与每任务黄金构建器。

Human-eval 校准。每个 LLM 判官维度都应当对一份小规模人工判官数据集校准 (50~100 份报告/维度)。当前运行已有 `HUMAN_URL_AUDIT.md` (每引文人工评级 30 报告样本); 把同一协议扩展到 `checklist`、`citation_alignment`、`presentation` 等维度, 会给 LLM 判官加上明确的误差棒。

推理模式对比。DeepSeek V4 thinking on、GPT-5 extended thinking、Claude with reasoning, 这三类推理模式都产生不同的输出模式, 值得显式比较。真实性门反转的发现在推理模型上可能更尖锐, 因为推理模型在文献中被记录有「更自信地捏造」的倾向。

长程任务。当前任务的墙钟上限是 1800 秒/智能体。需要 4 小时以上「搜-取-写」循环并带检查点的任务, 会测试「长程一致性」这一现基准无法覆盖的能力轴。

参考文献

- [1] G. Mialon, C. Fourrier, T. Wolf, et al. GAIA: A Benchmark for General AI Assistants. *arXiv preprint*, 2023. <https://arxiv.org/abs/2311.12983>
- [2] J. Wei, Z. Hou, S. Bowman, et al. BrowseComp: A Simple yet Challenging Benchmark for Browsing Agents. OpenAI Technical Report, 2024.
- [3] O. Yoran, S. Amouyal, C. Malaviya, et al. AssistantBench: Can Web Agents Solve Realistic and Time-Consuming Tasks? *arXiv preprint*, 2024. <https://arxiv.org/abs/2407.15711>
- [4] A. Drouin, M. Gasse, M. Caccia, et al. BrowserGym: A Unified Sandbox for Web-Agent Evaluation. *arXiv preprint*, 2024.
- [5] S. Zhou, F. F. Xu, H. Zhu, et al. WebArena: A Realistic Web Environment for Building Autonomous Agents. *Proceedings of the International Conference on Learning Representations (ICLR)*, 2024.
- [6] X. Liu, H. Yu, H. Zhang, et al. AgentBench: Evaluating LLMs as Agents. *Proceedings of the International Conference on Learning Representations (ICLR)*, 2024.
- [7] T. Gao, H. Yen, J. Yu, et al. Enabling Large Language Models to Generate Text with Citations. *Proceedings of EMNLP*, 2023.
- [8] S. Min, K. Krishna, X. Lyu, et al. FactScore: Fine-grained Atomic Evaluation of Factual Precision in Long Form Text Generation. *Proceedings of EMNLP*, 2023.
- [9] S. Es, J. James, L. Espinosa-Anke, et al. RAGAS: Automated Evaluation of Retrieval Augmented Generation. *arXiv preprint*, 2023. <https://arxiv.org/abs/2309.15217>
- [10] L. Zheng, W.-L. Chiang, Y. Sheng, et al. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. *Proceedings of NeurIPS Datasets and Benchmarks Track*, 2024.
- [11] W.-L. Chiang, L. Zheng, Y. Sheng, et al. Chatbot Arena: An Open Platform for Evaluating LLMs by Human Preference. *Proceedings of the International Conference on Machine Learning (ICML)*, 2024.
- [12] R. A. Bradley and M. E. Terry. Rank Analysis of Incomplete Block Designs: I. The Method of Paired Comparisons. *Biometrika*, 39(3/4): 324-345, 1952.
- [13] A. E. Elo. *The Rating of Chess Players, Past and Present*. Arco Publishing, 1978.

- [14] T. Gao, H. Yen, et al. ReClaim: Calibrated NLI Judges for Cited Claim Verification. *arXiv preprint*, 2024.
- [15] Z. Ji, H. Liu, K. Chen, et al. DRACO: Decomposed Rubric Aggregation for LLM-as-Judge. *arXiv preprint*, 2026.
- [16] G. Li, H. Hammoud, H. Itani, et al. CAMEL: Communicative Agents for "Mind" Exploration of Large Language Model Society. *Proceedings of NeurIPS*, 2023. <https://github.com/camel-ai/camel>
- [17] A. Roucher, T. Wolf, et al. **smolagents**: A Lightweight Toolcalling Agent Library. HuggingFace, 2024. <https://github.com/huggingface/smolagents>
- [18] A. Elovic, et al. **gpt-researcher**: An Autonomous Agent for Comprehensive Online Research. 2023. <https://github.com/assafelovic/gpt-researcher>
- [19] H. Chase, et al. LangChain: Building applications with LLMs through composability. 2022. <https://github.com/langchain-ai/langchain>
- [20] bytedance/DeerFlow. DeerFlow: Multi-agent deep research framework on langgraph. 2024. <https://github.com/bytedance/deer-flow>
- [21] LearningCircuit. local-deep-research: Programmatic deep research with local control. 2024. <https://github.com/LearningCircuit/local-deep-research>
- [22] Intelligent-Internet. ii-researcher: ReAct chain-of-search deep research agent. 2024. <https://github.com/Intelligent-Internet/ii-researcher>
- [23] qx-labs. agents-deep-research: openai-agents SDK based deep research framework. 2024. <https://github.com/qx-labs/agents-deep-research>
- [24] Y. Shao, Y. Jiang, T. Kanell, et al. STORM: Assisting in Writing Wikipedia-like Articles From Scratch with Large Language Models. *Proceedings of NAACL*, 2024. <https://github.com/stanford-oval/storm>
- [25] Alibaba-NLP. Tongyi DeepResearch: Qwen-based deep research with ReAct loop. 2024. <https://github.com/Alibaba-NLP/DeepResearch>
- [26] LangChain. deepagents: langgraph super-agent with planning and sub-agent spawning. 2024. <https://github.com/langchain-ai/deepagents>
- [27] dzhng. deep-research: TypeScript open-source deep research framework. 2024. <https://github.com/dzhng/deep-research>

A 复现命令

A.1 沙盒拉起 (在 westd 上)

```
ssh westd
wsl -d Ubuntu
cd /opt/deep_research

# 拉起三个 docker 语料 (Magento + Postmill + Kiwix)
docker compose -f infra/sandbox.docker-compose.yml up -d

# 端口检查
ss -tlnp | grep -E ':7770|:9999|:8090'

# 启动 Tavily shim (Windows 计划任务)
schtasks /Run /TN "ShimDaemon"

# 启动 DS proxy
schtasks /Run /TN "DsProxy"

# 健康检查
curl -fsS http://localhost:8081/health
curl -fsS http://localhost:8088/v1/models
```

A.2 跑一组 agent x task 并打分

```
.venv-camel/bin/python3 scripts/run_deep_task.py \
    --agent camel-ai --task dr_cross_deep_0001 \
    --backbone deepseek-v4-flash --out-suffix matrix

.venv-camel/bin/python3 scripts/score_deep_answer.py \
    --task dr_cross_deep_0001 \
    --answer data/results/deep/camel-ai__dr_cross_deep_0001_matrix
    .md \
    --out data/results/deep/camel-ai__dr_cross_deep_0001_matrix
    .score.json
```

A.3 重建排行榜

```
.venv-camel/bin/python3 scripts/build_deep_leaderboard.py
# reads data/results/deep_v3/*_matrix.score.json
# writes data/results/deep_v3/LEADERBOARD_DEEP.md
# writes data/results/deep_v3/leaderboard_deep.json
```

A.4 批量并行运行 (6 个 worker)

```
bash scripts/parallel_bulk_launch.sh 6
```

A.5 任一智能体运行所需的环境变量

```
DS_PROXY_URL=http://localhost:8088/v1
SHIM_URL=http://localhost:8081
OPENAI_API_KEY=anything
JUDGE_BASE_URL=http://localhost:8088/v1
JUDGE_MODEL=deepseek-v4-flash
SHOPPING=http://localhost:7770
REDDIT=http://localhost:9999
WIKIPEDIA=http://localhost:8090
TAVILY_API_KEY=tvly-shim-fake
```

A.6 虚拟环境对照表

B 术语表

ALCE alignment。Gao et al. (2023) 引入的「每引文精确率/召回率」度量: 对每一个 (claim, cited_url) 对, 被引页面是否真的支撑了 claim? 精确率为「被支撑对数 / 总对数」, 召回率为「至少有一个被支撑引文的句子数 / 有任意引文的句子数」。实现位于 `src/verifiers/citation_alignment_verifier.py`。

Bradley-Terry。Bradley & Terry (1952) 的成对比较模型: 给定大量成对结果, 为每个竞争者拟合单一评分, 使「A 击败 B 的概率」= $\sigma(R_A - R_B)$ 。Elo 更新是该似然 MLE 的迭代算法。实现位于 `src/scoring/arena.py`。

composite_v1。legacy 加性复合分: $Q = 0.40 \cdot \text{url_cov} + 0.40 \cdot \text{checklist} + 0.20 \cdot \text{spec}$, 无可达性门。仅用于演示真实性门反转。

虚拟环境	Python	使用方
<code>.venv-camel</code>	3.11	camel-ai, flowsearcher-ds, 打分/判官, 排行榜
<code>.venv-smol</code>	3.11	smolagents
<code>.venv-gptr</code>	3.11	gpt-researcher
<code>.venv-storm</code>	3.11	storm + co-storm
<code>.venv-langchain-odr</code>	3.11	langchain-open_deep_research (LangChain 1.x)
<code>.venv-ldr312</code>	3.12	ldr
<code>.venv-lcdr</code>	3.11	langchain-ai/local-deep-researcher
<code>.venv-ii</code>	3.11	ii-researcher (子进程)
<code>.venv-qx</code>	3.11	qx-agents
<code>.venv-tongyi</code>	3.11	Tongyi DeepResearch
<code>deer-flow-v1/.venv</code>	3.12	DeerFlow
Node + <code>.venv-camel</code>	Node 20	dzhng

composite_v2。头部排名。 $\text{reach} \cdot Q$ 。可达性门是乘性的: $\text{reach} = 0$ 强制复合分归零, 无论报告读起来多好。

composite_v3。七维松弛复合分。 $\max(0.1, \text{reach}) \cdot \text{raw}$, 其中 **raw** 是 `url_coverage`、`quote_match`、`checklist`、`spec`、`citation_alignment`、`analysis_depth`、`presentation` 的加权和。用于运行级审计, 不驱动头部 Elo。

Degenerate run / 退化样本。被 `_looks_degenerate` (`scripts/build_deep_leaderboard.py:82-131`) 剔除的打分行。四个模式:`short-answer-no-signal`、`sandbox infra failure`、`judge total failure`、`runner-placeholder text`。从 Bradley-Terry 对战中被排除以保持评分干净。

ds_proxy。`localhost:8088` 上的 OpenAI 兼容 HTTP 服务 (`integrations/ds_proxy/app.py`)。把每次 LLM 调用代理给 DeepSeek V4-flash, 注入 `thinking: disabled`, 服务端持有真密钥。所有智能体与所有 LLM 判官都打到同一份代理。

Golden pool / 黄金池。每任务一份 `data/golden/deep/dr_cross_deep_XXXX.json`, 含 `must_cite_urls` (智能体应当命中的 URL)、`expected_pool_urls` (称职智能体应当找到的更广泛语料)、`triples` (主-谓-宾 + `source_url` + `quoted_span`, 驱动 NLI)。冻结, 一次性爬取, 从不重生成。

must-cite。黄金 `must_cite_urls` 列表中的 URL, 带权重。URL 覆盖的主项是 `must-cite` 召回率, 按 per-URL 权重加权。

Permutation p-value / 置换 p 值。「两个智能体等强」零假设下观察到至少这么大 Elo 差距的概率。在相邻名次对内 shuffle 对战结果 1000 次估计。

Reachability gate / 可达性门。`composite_v2` 中的乘性可达因子。智能体引用的 URL 但凡不返 200 都会拉低可达性, $\text{reach} = 0$ 直接归零复合分。

Shim。localhost:8081 上的 Tavily 兼容搜索服务 (integrations/search_shim/app.py)。每个框架的 Tavily 客户端都指向这个端点。shim 把查询路由到三个沙盒索引并返 Tavily 形态 JSON。

Spec / 规范。Markdown 规范标志的通过率, 默认包含 words_ok、citations_ok、paragraphs_ok。spec 的取值是这三个布尔的均值。

Pillar / 维度。一份打分文件中的一个评分项。本基准有 7 个 pillar:URL 覆盖、URL 可达、引文文本匹配、claim NLI、checklist、引文对齐、表达 (含分析深度)。